2a2ec4ff-e814-41ff-ab64-7e67a68e597f

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'micronaut-projects/micronaut-sql' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/micronaut-sql/micronaut-sql (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-15T15:32:26.797Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix Vert.x PG PEM trust option configuration
</pr_title>

<pr_description>
Summary
- add explicit PEM trust option binding for the Vert.x PostgreSQL client
- resolve effective PG SSL options before pool creation so CA trust config is preserved
- document the configuration path and add focused regression coverage

Verification

- ./gradlew :micronaut-vertx-pg-client:test --tests 'io.micronaut.configuration.vertx.pg.client.PgClientConfigurationSpec'

Resolves #985
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@graemerocher</author>
@copilot review
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot review

Reviewed (2d89d9b). Changes made as a result:

- **`PgConnectOptionsResolver.applyNetClientOptions` removed** — the 7-branch field-by-field SSL merge was over-engineered, had no test coverage, and introduced new behaviour (propagating `NetClientOptions` SSL settings into `connectOptions`) beyond the stated fix. `netClientOptions` is still passed separately to `createPool` as before; only the PEM trust cert paths are now applied to `connectOptions` via `applyPemTrustOptions`.
- **Class-level Javadoc added** to `PgConnectOptionsResolver` explaining its role.
- **Copyright year corrected** in new files to `2017-2020` (matching the rest of the module).

Everything else looks good: Checkstyle, Spotless, and all 3 tests pass.

> [!WARNING]
>
>

</comment_old>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

> [!NOTE]
> Copilot was unable to run its full agentic suite in this review.

This PR fixes Vert.x PostgreSQL SSL “verify-ca” behavior by ensuring PEM trust options are bound and preserved when building the effective `PgConnectOptions` (including URI-based configuration), and adds documentation plus regression tests.

Changes:
- Add `pem-trust-options` configuration binding and apply it when resolving effective `PgConnectOptions`.
- Centralize connect options resolution for both URI and property configuration paths before pool creation.
- Add regression tests for “verify-ca” with PEM trust and document the configuration.

Reviewed changes

Copilot reviewed 7 out of 7 changed files in this pull request and generated 4 comments.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>vertx-pg-client/src/test/groovy/io/micronaut/configuration/vertx/pg/client/PgClientConfigurationSpec.groovy:91-98</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`connectFailure` can throw `TimeoutException` and

`InterruptedException` from `Future#get(...)` but only catches `ExecutionException`. This can make the test fail for the wrong reason (e.g., timeouts under load) and, in the interrupted case, can swallow the interrupt. Handle `TimeoutException` (return it or unwrap as appropriate) and catch `InterruptedException` while restoring the thread interrupt flag (`Thread.currentThread().interrupt()`), then return the exception so assertions can be explicit.
</comment_old>

</comment_thread>

<comment_thread>
<file>vertx-pg-client/src/test/groovy/io/micronaut/configuration/vertx/pg/client/PgClientConfigurationSpec.groovy:58-64</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The test relies on a hard-coded port (`65432`) being closed to produce a `ConnectException`. This can be flaky in CI/dev environments if something is listening on that port. Prefer deterministically selecting a free port (e.g., bind a `ServerSocket(0)` to get an ephemeral free port, capture it, then close it before attempting the connection) so the test reliably exercises the intended failure path.
</comment_old>

</comment_thread>

<comment_thread>
<file>src/main/docs/guide/vertxpgclient.adoc:41</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The docs use `ssl-mode: verify-ca` while the new regression tests use `VERIFY_CA`. Even if the binder accepts multiple formats, this inconsistency can confuse users. Consider standardizing on a single documented value/format (and optionally mention accepted aliases/case-insensitivity if applicable).
```suggestion
      ssl-mode: VERIFY_CA
```
</comment_old>

</comment_thread>

<comment_thread>
<file>vertx-pg-client/src/main/java/io/micronaut/configuration/vertx/pg/client/PgClientConfiguration.java:65-71</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The Javadoc has an empty leading line (`*` on line 66). Remove the blank line to keep Javadoc formatting consistent (start the description right away, or add a real summary sentence).
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4253173039</comment_id>
<author>@graemerocher</author>
@copilot apply changes based on the comments in this thread
</comment_new>

</pr_comments>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.

* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

All four review items have been addressed in commit `3b520bd`:

Review item	Change
Hard-coded port `65432`	Both SSL tests now use `findFreePort()` via `ServerSocket(0)` to get a deterministically free ephemeral port
`connectFailure` missing `TimeoutException` / `InterruptedException`	Both are now caught; `InterruptedException` restores the interrupt flag with `Thread.currentThread().interrupt()`
`ssl-mode: verify-ca` in docs	Changed to `VERIFY_CA` to match the Vert.x enum name and test values
Empty `*` line in `getNetClientOptions()` Javadoc	Blank leading line removed

All 3 tests pass; Checkstyle and Spotless are clean.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'micronaut-projects/micronaut-sql' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/micronaut-sql/micronaut-sql (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-17T13:32:52.928Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix Vert.x PG PEM trust option configuration
</pr_title>

<pr_description>
Summary
- add explicit PEM trust option binding for the Vert.x PostgreSQL client
- resolve effective PG SSL options before pool creation so CA trust config is preserved
- document the configuration path and add focused regression coverage

Verification
- ./gradlew :micronaut-vertx-pg-client:test --tests 'io.micronaut.configuration.vertx.pg.client.PgClientConfigurationSpec'

Resolves #985
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@graemerocher</author>
@copilot review
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot review

Reviewed (2d89d9b). Changes made as a result:

- **`PgConnectOptionsResolver.applyNetClientOptions` removed** — the 7-branch field-by-field SSL merge was over-engineered, had no test coverage, and introduced new behaviour (propagating `NetClientOptions` SSL settings into `connectOptions`) beyond the stated fix. `netClientOptions` is still passed separately to `createPool` as before; only the PEM trust cert paths are now applied to `connectOptions` via `applyPemTrustOptions`.
- **Class-level Javadoc added** to `PgConnectOptionsResolver` explaining its role.
- **Copyright year corrected** in new files to `2017-2020` (matching the rest of the module).

Everything else looks good: Checkstyle, Spotless, and all 3 tests pass.

> [!WARNING]
>
>

</comment_old>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

> [!NOTE]
> Copilot was unable to run its full agentic suite in this review.

This PR fixes Vert.x PostgreSQL SSL “verify-ca” behavior by ensuring PEM trust options are bound and preserved when building the effective `PgConnectOptions` (including URI-based configuration), and adds documentation plus regression tests.

Changes:
- Add `pem-trust-options` configuration binding and apply it when resolving effective `PgConnectOptions`.
- Centralize connect options resolution for both URI and property configuration paths before pool creation.

- Add regression tests for “verify-ca” with PEM trust and document the configuration.

Reviewed changes

Copilot reviewed 7 out of 7 changed files in this pull request and generated 4 comments.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread_resolved>
<file>vertx-pg-client/src/test/groovy/io/micronaut/configuration/vertx/pg/client/PgClientConfigurationSpec.groovy:102-114</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`connectFailure` can throw `TimeoutException` and `InterruptedException` from `Future#get(...)` but only catches `ExecutionException`. This can make the test fail for the wrong reason (e.g., timeouts under load) and, in the interrupted case, can swallow the interrupt. Handle `TimeoutException` (return it or unwrap as appropriate) and catch `InterruptedException` while restoring the thread interrupt flag (`Thread.currentThread().interrupt()`), then return the exception so assertions can be explicit.
</comment_old>

</comment_thread_resolved>

<comment_thread_resolved>
<file>vertx-pg-client/src/test/groovy/io/micronaut/configuration/vertx/pg/client/PgClientConfigurationSpec.groovy:62-68</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The test relies on a hard-coded port (`65432`) being closed to produce a `ConnectException`. This can be flaky in CI/dev environments if something is listening on that port. Prefer deterministically selecting a free port (e.g., bind a `ServerSocket(0)` to get an ephemeral free port, capture it, then close it before attempting the connection) so the test reliably exercises the intended failure path.
</comment_old>

</comment_thread_resolved>

<comment_thread_resolved>
<file>src/main/docs/guide/vertxpgclient.adoc:41</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The docs use `ssl-mode: verify-ca` while the new regression tests use `VERIFY_CA`. Even if the binder accepts multiple formats, this inconsistency can confuse users. Consider standardizing on a single documented value/format (and optionally mention accepted aliases/case-insensitivity if applicable).
```suggestion
      ssl-mode: VERIFY_CA
```
</comment_old>

</comment_thread_resolved>

<comment_thread_resolved>
<file>vertx-pg-client/src/main/java/io/micronaut/configuration/vertx/pg/client/PgClientConfiguration.java:65-70</file>
<original_commit>2d89d9b</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The Javadoc has an empty leading line (`*` on line 66). Remove the blank line to keep Javadoc formatting consistent (start the description right away, or add a real summary sentence).
</comment_old>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@graemerocher</author>
@copilot apply changes based on the comments in <a
href="https://github.com/micronaut-projects/micronaut-sql/pull/
1910#pullrequestreview-4114578205">this thread
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot apply changes based on the comments in <a
href="https://github.com/micronaut-projects/micronaut-sql/pull/
1910#pullrequestreview-4114578205">this thread

Done in 3b520bd. Applied all four review comments:

- **Dynamic port**: both SSL tests now bind a `ServerSocket(0)`
to obtain a free ephemeral port, eliminating any risk of
collision with a real service on `65432`.
- **Exception handling**: `connectFailure` now catches
`TimeoutException` (returned as-is) and `InterruptedException`
(interrupt flag restored via
`Thread.currentThread().interrupt()`, then returned).
- **Docs casing**: `ssl-mode: verify-ca` → `ssl-mode: VERIFY_CA`
to match the Vert.x enum name and the test values.
- **Javadoc**: removed the blank leading `*` line from
`getNetClientOptions()`.

> [!WARNING]
>
>

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>vertx-pg-client/src/test/groovy/io/micronaut/
configuration/vertx/pg/client/
PgClientConfigurationSpec.groovy:57</file>
<original_commit>3b520bd</original_commit>

<comment_new>
<comment_id>3091968148</comment_id>
<author>@radovanradic</author>
Would it make more sense to do actual verification for pem trust options, something like
```
void "test vertx-pg-client connects with direct options when verify-ca trust options are configured"() {
        given:
        int port = findFreePort()
        when:
        ApplicationContext applicationContext = ApplicationContext.run(
                'vertx.pg.client.host': 'localhost',
                'vertx.pg.client.port': port,
                'vertx.pg.client.ssl': true,
                'vertx.pg.client.ssl-mode': 'VERIFY_CA',
                'vertx.pg.client.pem-trust-options.cert-paths[0]': 'certs/ca.crt'
        )
        PgConnectOptions options = PgConnectOptionsResolver.resolve(

applicationContext.getBean(PgClientConfiguration),

applicationContext.getBean(PgPemTrustOptionsConfiguration)
        )
        PemTrustOptions trustOptions = (PemTrustOptions) options.sslOptions.trustOptions

        then:
        options.host == 'localhost'
        options.port == port
        options.sslMode == SslMode.VERIFY_CA
        options.sslOptions != null
        trustOptions.certPaths == ['certs/ca.crt']

        cleanup:
        applicationContext?.stop()
    }
```
And the other test as well
```
void "test vertx-pg-client uri mode keeps verify-ca trust options during connect"() {
        given:
        int port = findFreePort()
```

```
        when:
        ApplicationContext applicationContext =
ApplicationContext.run(
                'vertx.pg.client.uri': "postgresql://
user:secret@localhost:${port}/the-db",
                'vertx.pg.client.ssl': true,
                'vertx.pg.client.ssl-mode': 'VERIFY_CA',
                'vertx.pg.client.pem-trust-options.cert-
paths[0]': 'certs/ca.crt'
        )
        PgConnectOptions options =
PgConnectOptionsResolver.resolve(

applicationContext.getBean(PgClientConfiguration),

applicationContext.getBean(PgPemTrustOptionsConfiguration)
        )
        PemTrustOptions trustOptions = (PemTrustOptions)
options.sslOptions.trustOptions

        then:
        options.host == 'localhost'
        options.port == port
        options.database == 'the-db'
        options.user == 'user'
        options.password == 'secret'
        options.sslMode == SslMode.VERIFY_CA
        options.sslOptions != null
        trustOptions.certPaths == ['certs/ca.crt']

        cleanup:
        applicationContext?.stop()
    }
```

which seems to fail as ssl options are not being set
properly (which is issue probably existed before this PR)
</comment_new>
</comment_thread>

<comment_thread>
<file>vertx-pg-client/src/main/java/io/micronaut/configuration/
vertx/pg/client/PgPemTrustOptionsConfiguration.java:30</file>
<original_commit>3b520bd</original_commit>

<comment_new>
<comment_id>3091969559</comment_id>
<author>@radovanradic</author>
```suggestion
 * @since 7.0.0
```
</comment_new>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4268263993</comment_id>
<author>@graemerocher</author>
@copilot resolve the merge conflicts in this pull request
</comment_new>

</pr_comments>

</comments>

The last **3** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.